Stockify - Paper Trading with Real-Time Market Data

AN INDUSTRIAL ORIENTED MINI PROJECT REPORT

Submitted

in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

Janagam Shashank 23B81A05MS

Dasari Sai Teja 23B81A05MJ

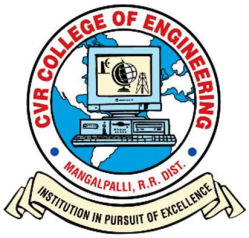
Madugula Jashwanth Reddy 23B81A05LK

Under the Guidance of

Dr. C. Ramesh

Professor, Department of CSE and

Associate Dean, Student Affairs



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CVR COLLEGE OF ENGINEERING

(***An Autonomous institution, NBA, NAAC Accredited and Affiliated to JNTUH, Hyderabad***)

Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Rangareddy (D), Telangana- 501 510

April-2026

CVR COLLEGE OF ENGINEERING

(An UGC Autonomous Institution with NAAC 'A' Grade, Affiliated to JNTUH)
Vastunagar, Mangalpalli (V), Ibrahimpatan (M), R.R. District, Telangana

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Industrial Oriented Mini project entitled "**Stockify - Paper Trading with Real-Time Market Data**" being submitted by Janagam Shashank 23B81A05MS, Dasari Sai Teja 23B81A05MJ, Madugula Jashwanth Reddy 23B81A05LK in partial fulfillment for the award of Bachelor of Technology in Computer Science and Engineering to the CVR College of Engineering, is a record of bona fide work carried out by them under my guidance and supervision during the year 2025-2026.

The results embodied in this Industrial Oriented Mini project work have not been submitted to any other University or Institute for the award of any degree or diploma.

Signature of the project guide

Dr. C. Ramesh
Professor, Department of CSE and
Associate Dean, Student Affairs

Signature of the HOD

Department Of CSE

External Examiner

DECLARATION

We hereby declare that this project report titled **"Stockify – Paper Trading with Real-Time Market Data"** submitted to the Department of Computer Science and Engineering, CVR College of Engineering, is a record of original work done by us. The information and data presented in this report are authentic to the best of our knowledge. This Industrial Oriented Mini Project report has not been submitted to any other university or institution for the award of any degree or diploma, nor has it been published at any time before.

Janagam Shashank 23B81A05MS

Dasari Sai Teja 23B81A05MJ

Madugula Jashwanth Reddy 23B81A05LK

Date:

Place: Hyderabad

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal, **Dr. K. Ramamohan Reddy**, the Head of the Department, **Mrs. Vani Vathsala**, and our project guide, **Dr. C. Ramesh**, for their continuous support, valuable guidance, and encouragement throughout the development of this Industrial Oriented Mini Project titled **"Stockify – Paper Trading with Real-Time Market Data."**

Their technical expertise, insightful suggestions, and constant motivation played a crucial role in shaping this project from the initial stages of requirement analysis to design, implementation, and final testing. Their mentorship not only helped us in successfully completing the project but also enhanced our technical knowledge and problem-solving abilities.

We would also like to extend our heartfelt thanks to all the faculty members of the Department of Computer Science and Engineering for their guidance, encouragement, and valuable inputs during the course of this project. We are equally thankful to our peers for their cooperation, constructive discussions, and support, which contributed significantly to the improvement of our work.

Finally, we thank the **Department of Computer Science and Engineering**, CVR College of Engineering, for providing the necessary resources and infrastructure. The department's commitment to research and innovation greatly contributed to the success of this project.

ABSTRACT

Stockify is a full-stack paper trading platform designed to help users learn and practice stock trading in a safe, risk-free environment. Instead of using real money, users can explore the market, track live price movements, and place buy or sell orders using virtual funds. The main idea behind the project is to make stock trading easier to understand, especially for beginners who often find real markets complex and intimidating. By bringing everything—stock search, real-time price tracking, portfolio management, and transaction history—into one simple and interactive dashboard, **Stockify** provides a clear and practical way to understand how trading actually works in real life. This platform focuses on creating an experience that closely resembles real-world trading while remaining simple and user-friendly. Features like real-time updates ensure users stay aware of market changes, while AI-based insights help them interpret trends and make better decisions. Users can experiment with different trading strategies, learn from their successes and mistakes, and gradually build confidence without any financial risk. This makes Stockify not just a tool for simulation, but also a valuable learning platform for students and aspiring investors. Overall, **Stockify** demonstrates a practical and scalable solution that combines learning with real-time simulation. It improves accessibility to stock market education and provides a strong foundation for understanding trading concepts. The system is designed to be reliable and production-ready, with the ability to be deployed using cloud technologies such as AWS EC2 for backend services and AWS CloudFront for efficient and fast content delivery, ensuring a smooth and responsive user experience.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

SYMBOLS

Symbol	Meaning
API	Application Programming Interface endpoint notation
JWT	JSON Web Token — encoded authentication credential
{id}	Route parameter placeholder in API path
HTTP 4xx	Client-side error response codes (401, 403, 404 etc.)
HTTP 2xx	Success response codes (200 OK, 201 Created)
ms	Milliseconds — unit for API response time measurement
DXA	Document XML units (1440 DXA = 1 inch) — internal Word measure
fps	Frames per second — canvas rendering performance metric
GB / MB	Gigabytes / Megabytes — storage and memory units

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
App Router	Application Router — Next.js routing paradigm
AWS	Amazon Web Services
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
CSE	Computer Science and Engineering
CSS	Cascading Style Sheets
DOM	Document Object Model
GPU	Graphics Processing Unit
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
OAuth	Open Authorization
PPTX	PowerPoint Presentation file format
PWA	Progressive Web Application
RPC	Remote Procedure Call
SaaS	Software as a Service
SQL	Structured Query Language
NS	National Stock Exchange
BSE	Bombay Stock Exchange
TS	TypeScript
UI	User Interface
UX	User Experience
WAI-ARIA	Web Accessibility Initiative — Accessible Rich Internet Applications

CHAPTER 1: INTRODUCTION

1.1 MOTIVATION

The motivation behind developing **Stockify** comes from the difficulty many beginners face when trying to enter the stock market. Real trading platforms often involve financial risk, complex interfaces, and a steep learning curve, which can discourage students and new investors from gaining practical experience. Most learners rely only on theory or scattered resources, without having a safe environment to apply their knowledge. This creates a gap between understanding stock market concepts and actually experiencing how trading works in real time.

Stockify aims to bridge this gap by providing a **risk-free paper trading platform** where users can practice trading using virtual funds while still interacting with real market data. The goal is to build confidence, improve decision-making skills, and help users understand market behavior without the fear of losing money. By combining real-time updates, portfolio tracking, and AI-based insights in a single platform, the project makes learning more interactive and practical. Ultimately, the motivation is to make stock market education more accessible, engaging, and effective for beginners and aspiring traders.

1.2 PROBLEM STATEMENT

Many beginner and intermediate users find it difficult to understand and participate in stock market trading due to the risk of financial loss, lack of practical exposure, and the complexity of existing trading platforms. Most available tools either focus only on real trading or provide limited learning support, forcing users to switch between multiple platforms for stock analysis, portfolio tracking, and transaction management. This fragmented approach makes the learning process confusing and inefficient. Additionally, many learning platforms do not accurately replicate real trading conditions such as stop-loss execution, intraday trading constraints, and automatic square-off mechanisms, which are essential for understanding actual market behaviour.

There is a need for a unified system that not only allows users to practice trading in a risk-free environment but also closely simulates real-world trading scenarios. The system should support features like real-time market data, portfolio management, simulated buy/sell orders, stop-loss triggers, intraday trading rules, and automatic square-off at market close. By incorporating these realistic trading conditions along with AI-based insights and user-friendly interfaces, the platform can help users gain practical experience, improve decision-making skills, and better prepare for real stock market participation.

1.3 PROJECT OBJECTIVES

1. To design and develop a responsive and user-friendly frontend that enables efficient stock discovery, visualization of charts, and interactive user dashboards.
2. To build a scalable backend system that provides APIs for stock data retrieval, search functionality, order execution, portfolio management, and payment handling.
3. To implement secure authentication mechanisms that ensure protected access to user-specific features and maintain data privacy.
4. To enable real-time communication using WebSocket technology for instant stock price updates and improved user experience.
5. To integrate wallet functionality that supports fund addition, transaction tracking, and seamless financial operations within the platform.
6. To incorporate AI-based insight generation features that assist users in understanding market trends and making informed trading decisions.

1.4 SCOPE OF THE PROJECT

The scope of the **Stockify** project is to design and develop a web-based paper trading platform that allows users to simulate real stock market trading in a safe and controlled environment without using real money. The system provides functionalities such as stock search, real-time price monitoring, portfolio management, and execution of buy and sell orders using virtual funds. It aims to replicate real trading conditions by supporting features like stop-loss mechanisms, intraday trading behaviour, and automatic square-off, helping users gain practical experience and understand market dynamics effectively.

The platform also includes wallet management for handling virtual funds, transaction tracking, and performance analysis to evaluate user trading strategies. AI-based insights are integrated to assist users in interpreting market trends and making better decisions. Real-time communication ensures that stock data updates are reflected instantly, enhancing the simulation experience. The system is designed to be scalable and user-friendly, supporting deployment on cloud infrastructure such as AWS EC2, CloudFront, and Cloudflare for efficient performance, security, and availability. However, the project is limited to simulation purposes and does not involve real financial transactions or integration with actual trading exchanges.

1.5 PROJECT REPORT ORGANIZATION

This report is organized into six chapters, each addressing a distinct phase of the software development lifecycle:

- Chapter 2 presents the literature survey, reviewing existing stock market tools and identifying the gaps that **Stockify** addresses.
- Chapter 3 defines the Software Requirement Specification (SRS), including functional requirements, non-functional requirements, and system resource specifications.
- Chapter 4 presents the proposed system design through use case, class, activity, sequence, component, and deployment views along with the technology stack description.
- Chapter 5 discusses the implementation modules in detail and presents the testing strategy with representative test cases.
- Chapter 6 concludes the report with a summary of achievements and a roadmap of planned future enhancements.

CHAPTER 2: LITERATURE SURVEY

2.1 EXISTING WORK

The current landscape of financial technology platforms for retail investors can be broadly categorized into four major segments, each addressing specific aspects of trading and investment workflows.

2.1.1 Broker-Native Trading Applications

Platforms such as Groww, Zerodha Kite, and Upstox provide complete trading solutions with a primary focus on order execution, account management, and regulatory compliance [14]. These applications are efficient for real-time trading; however, their analytical capabilities and user guidance features are often limited, especially for beginners. Advanced insights and AI-based interpretation are generally not integrated.

2.1.2 Market Visualization Portals

Tools such as TradingView specialize in advanced charting and technical analysis with a wide range of indicators and visualization tools [12]. While these platforms are highly effective for analysing price trends and patterns, they do not maintain user-specific portfolio data, requiring users to manually track their investments across different platforms.

2.1.3 Financial Data Aggregators

Services such as Yahoo Finance provide comprehensive market data, news, and basic stock tracking features [1]. Although they offer broad coverage, the data may be delayed or limited in free versions, and these platforms do not support trade execution or personalized portfolio management

.

2.1.4 Portfolio Tracking Tools

Portfolio management platforms allow users to monitor investments, calculate returns, and analyse performance. However, these tools often lack integration with real-time data streams and trading functionalities, making it necessary for users to switch between multiple systems for complete workflow management.

2.2 LIMITATIONS OF EXISTING WORK

A detailed analysis of the above categories reveals several key limitations that impact user experience and efficiency:

Fragmented user experience: Users must rely on multiple platforms for stock discovery, analysis, trading, and portfolio tracking, leading to inefficiency and increased cognitive load.

Limited personalization: Most platforms provide generic data and watchlists without considering individual user portfolios, preferences, or trading behaviour.

Lack of actionable insights: While platforms provide extensive data and charts, they often fail to deliver meaningful interpretations or guidance, especially for non-expert users [13].

Insufficient learning support: Many systems are designed for experienced traders and do not provide adequate educational support or intuitive guidance for beginners.

Real-time performance challenges: Some platforms experience delays or reduced reliability in real-time data streaming during high traffic or peak market conditions.

Inference from Survey

From the above analysis, it is evident that there is a need for a unified platform that integrates real-time market data, portfolio tracking, simulated trading, and intelligent insights within a single system. A solution that combines these features with user-friendly design and AI-assisted recommendations can significantly enhance the learning experience and decision-making ability of retail investors.

CHAPTER 3: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

3.1 FUNCTIONAL REQUIREMENTS

FR-1: User Access and Session Handling

- The system shall support user login and authentication verification for all protected routes.
- Session-aware navigation shall direct authenticated users to the dashboard, funds, and portfolio views.
- Unauthenticated access to protected routes shall result in automatic redirection to the login page.

FR-2: Stock Data Retrieval

- The system shall fetch current stock quote data and historical price series for tracked symbols.
- The backend shall provide live market data streams for active screens via WebSocket and SSE protocols.
- Stock data shall be refreshed at configurable intervals to maintain near real-time accuracy.

FR-3: Search and Discovery

- Users shall be able to search for stocks by symbol or company name through a dynamic search bar.
- Search results shall update in real time as the user types, with debounced API calls to prevent excessive requests.

FR-4: Portfolio and Holdings

- The system shall display the user's current holdings, including quantity, average cost, current value, and profit/loss.
- Portfolio-level aggregated metrics such as total invested, current value, day's gain, and overall return shall be computed and displayed.

FR-5: Order Execution Flow

- Users shall be able to place buy and sell orders for supported stock symbols through the frontend.
- The backend order execution module shall validate inputs, process trade logic, and update the user's holdings accordingly.
- Order confirmation and failure responses shall be communicated clearly to the user.

FR-6: Payment and Wallet Management

- Users shall be able to add funds to their platform wallet through Razorpay payment gateway integration as virtual money in test mode.
- All fund additions and deductions shall be recorded as transactions retrievable in the transaction history view.
- The current wallet balance shall be reflected in real time across the platform.

FR-7: AI Insight Services

- The system shall expose AI insight endpoints that generate stock analysis summaries for selected symbols.
- Request throttling and rate limiting shall be enforced on AI routes to prevent abuse and control API costs.

FR-8: Compliance and Informational Content

- The platform shall display static policy pages including Privacy Policy, Terms of Service, Cookie Policy, and Disclaimer.

3.2 NON-FUNCTIONAL REQUIREMENTS

- **Performance**: Quote and search route responses shall be returned with low latency to ensure a responsive user experience.
- **Scalability**: The modular backend architecture shall support route-level extension without requiring changes to existing modules.
- **Security**: All protected routes shall enforce authentication middleware. Input validation, CORS controls, and secure cookie handling shall be implemented across all endpoints.
- **Reliability**: The application shall perform a database connectivity check on startup and handle connection failures gracefully.
- **Usability**: The frontend UI shall be fully responsive across desktop and mobile viewports, with intuitive navigation and clear feedback mechanisms.
- **Maintainability**: A clear separation of concerns between frontend components and backend modules shall be maintained, following established software engineering principles.
- **Availability**: The system shall be deployment-ready with Docker support to enable consistent environment configuration across development, staging, and production.

SOFTWARE AND HARDWARE REQUIREMENTS]

Software Requirements

- **Runtime Environment:**
  - **Node.js** (version 18 or higher) for executing backend services.
- **Package Manager:**
  - **npm** (Node Package Manager) for dependency management and project scripts.
- **Frontend Technologies:**
  - **React** 18 for building dynamic user interfaces.
  - **TypeScript** for type safety and better code maintainability.
  - **Vite** as a fast build and development tool.
  - **CSS** Modules for responsive and consistent UI design.
- **Backend Technologies:**
  - **Express.js** framework for handling API routes and business logic.
  - **Modular route architecture** for better code organization.
- **Database:**
  - **MongoDB** for storing user data, portfolio details, and transactions.
- **Cache Layer (Optional):**
  - **Redis** for caching frequently accessed data and improving performance.
- **Version Control:**
  - **Git** and **GitHub** for source code management and collaboration.
- **Browser Compatibility:**
  - Google Chrome (latest version)
  - Mozilla Firefox (latest version)
  - Microsoft Edge (latest version)

Deployment & Cloud Requirements

- **Cloud Platform:**
  - **Amazon Web Services (AWS)** for hosting and deployment.
- **Backend Hosting:**
  - **AWS EC2** instance for running backend server and APIs.
- **Frontend Hosting:**
  - **AWS CloudFront** for delivering frontend assets via CDN.
- **Security & CDN:**
  - **Cloudflare** for DDoS protection, caching, and improved performance.
- **Cloud Management Tools:**
  - **AWS CLI** for managing cloud resources and deployments.

    - **SSH client** for securely accessing EC2 instances.
- **AI Services:**
    - **Gemini API Model- gemini-2.5-flash-lite** for stock insights and recommendations
- **Additional Tools:**
    - Environment configuration using .env files for secure credentials.
    - Node process manager (optional, e.g., PM2) for running backend services continuously.

Hardware Requirements

- **Processor:** Intel i3 / Ryzen 3 or higher
- **RAM:** Minimum 8 GB (16 GB recommended)
- **Storage:** At least 10 GB free space (SSD preferred)
- **System Type:** 64-bit system
- **Display:** Minimum 1366 × 768 resolution
- **Network:** Stable internet connection
- **Input Devices:** Keyboard and mouse

3.4 SOFTWARE ARCHITECTURE

Stockify follows a three-tier web application architecture that cleanly separates concerns across the presentation, application, and data/integration layers:

- Presentation Layer: The React and TypeScript frontend delivers route-based pages and reusable UI components to the end user's browser.
- Application Layer: The Node.js and Express backend exposes a modular REST API surface handling stock data, search, order execution, portfolio analytics, authentication, payments, and AI insights.
- Data and Integration Layer: MongoDB provides persistent storage for users, orders, holdings, and transactions. External integrations include the Razorpay payment gateway, Yahoo Finance or equivalent market data APIs, and a generative AI provider for insight endpoints.

CHAPTER 4: PROPOSED SYSTEM DESIGN

4.0 PROPOSED METHODS

The proposed implementation of **Stockify** adopts a modular full-stack approach to ensure scalability, efficiency, and real-time performance. The system provides a responsive and interactive user interface for stock exploration, portfolio tracking, and simulated trading, while the backend efficiently manages API requests and business logic. MongoDB is used to store user data, portfolio details, transactions, and order records. Real-time stock updates are achieved through WebSocket communication, allowing continuous streaming of market data to users with minimal delay. The backend integrates external stock market data providers such as Yahoo Finance to fetch and process real-time information for visualization and analysis [1]. This ensures that users receive accurate and up-to-date market data, closely simulating real trading conditions similar to modern platforms like Groww [14].

Security and user management are implemented using token-based authentication, ensuring that only authorized users can access sensitive features such as portfolio tracking and trading operations. The order execution module processes buy and sell requests by validating user balance and updating holdings accordingly. Additionally, payment gateway integration such as Razorpay enables wallet funding and transaction tracking within the system [2]. AI-based insight services are incorporated to provide users with meaningful stock analysis and recommendations, supported by financial learning concepts from platforms like Investopedia [13], thereby improving decision-making capabilities and aligning with user-centric trading experiences.

Furthermore, the system emphasizes usability and performance through an intuitive interface, efficient data handling, and real-time responsiveness. Testing is carried out at multiple levels, including unit, integration, and end-to-end testing, to ensure system reliability and stability. The application is designed for scalable deployment using cloud infrastructure, ensuring high availability and smooth performance even under increased user load. Overall, the implementation demonstrates a practical and user-focused solution that closely aligns with real-world trading platforms while maintaining a safe and educational environment.

4.1 USE CASE DIAGRAM

The use case diagram of **Stockify** illustrates how different actors interact with the system to perform various functionalities. The primary actor, the user (investor), can register, log in, manage their profile, search for stocks, view real-time market data, and perform trading operations such as placing buy or sell orders and managing their portfolio. The system also integrates external actors like the payment gateway for handling wallet transactions and the stock data provider for delivering live market data. Additionally, an admin actor is responsible for managing users and monitoring system activities. The diagram highlights key features such as AI-based insights, notifications, and report generation, showing how all components work together to provide a seamless and efficient stock trading and analysis experience.

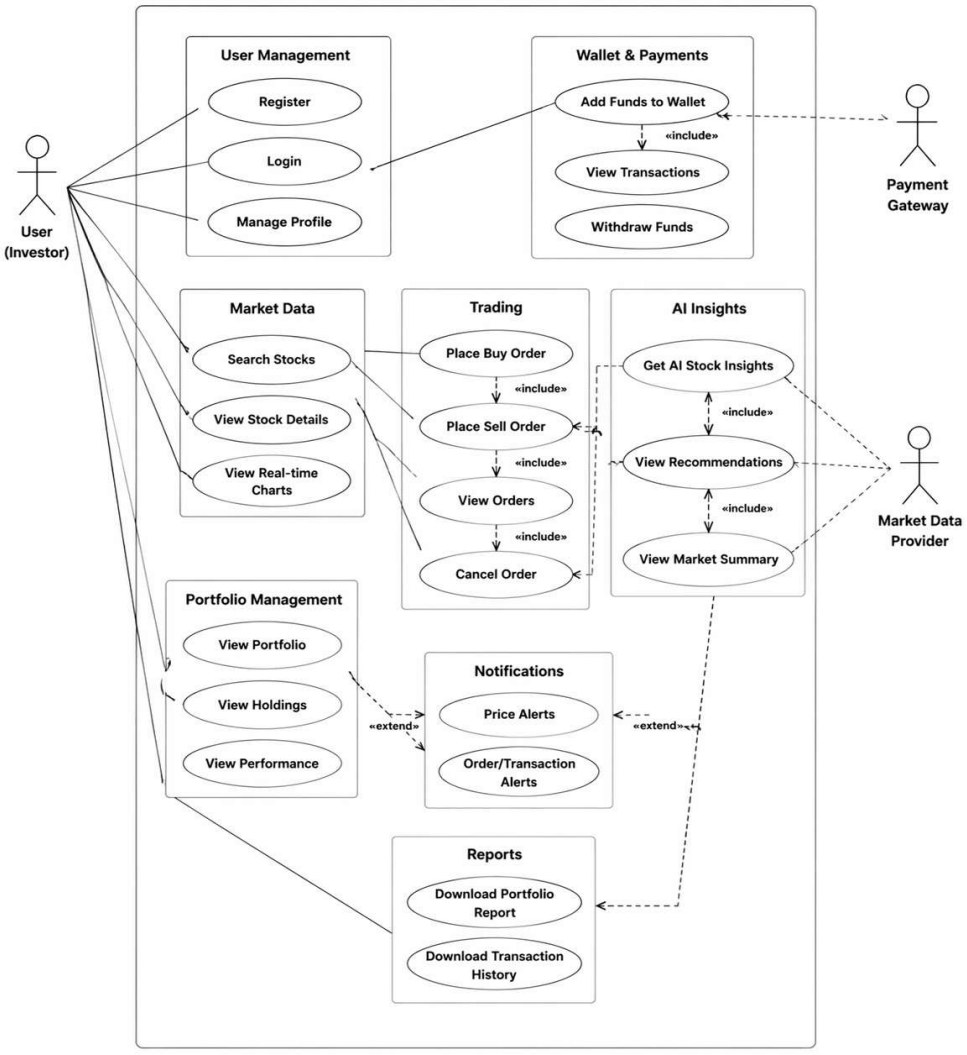


Figure 4.1.1 Use Case Diagram

4.2 CLASS DIAGRAM

The class diagram of **Stockify** represents the structural design of the system with key entities such as User, Portfolio, Holding, Stock, Order, Transaction, PaymentOrder, and AIInsight. The User class acts as the central entity associated with portfolio management, transactions, and order execution.

The Portfolio aggregates multiple holdings, each linked to a specific stock. Orders are processed to update holdings, while transactions track financial activities.

PaymentOrder manages wallet funding, and AIInsight provides analytical recommendations for stocks. This design ensures modularity, scalability, and clear separation of responsibilities across system components.

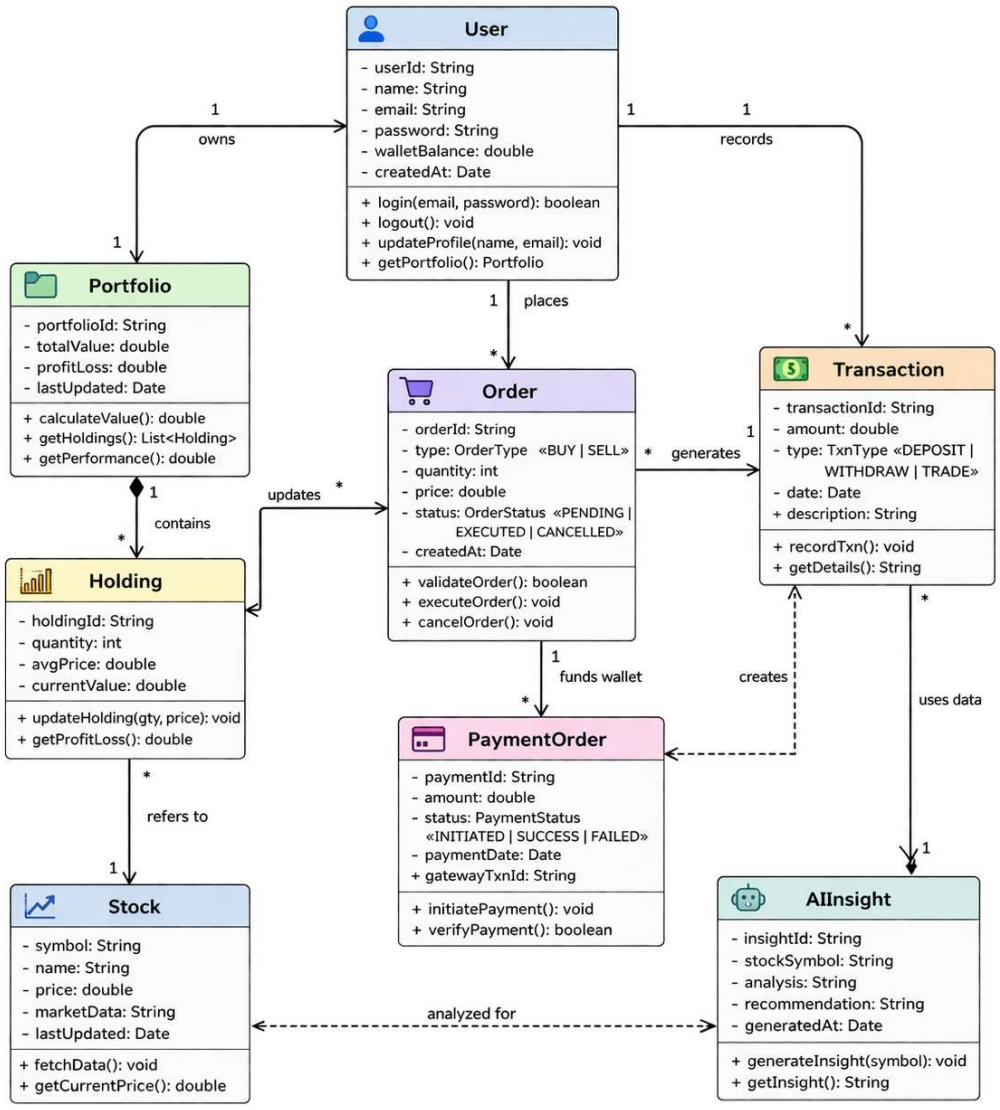


Figure 4.2.1 Class Diagram

4.3 ACTIVITY DIAGRAM

The activity diagram of **Stockify** illustrates the step-by-step workflow of the system starting from user login to logout. The process begins when the user launches the application and successfully logs in, after which the dashboard is displayed with multiple options such as searching stocks, viewing portfolio, placing buy/sell orders, managing wallet transactions, and requesting AI insights. Each user action triggers system processes like fetching real-time stock data, validating orders, processing payments through external gateways, and generating insights using AI services.

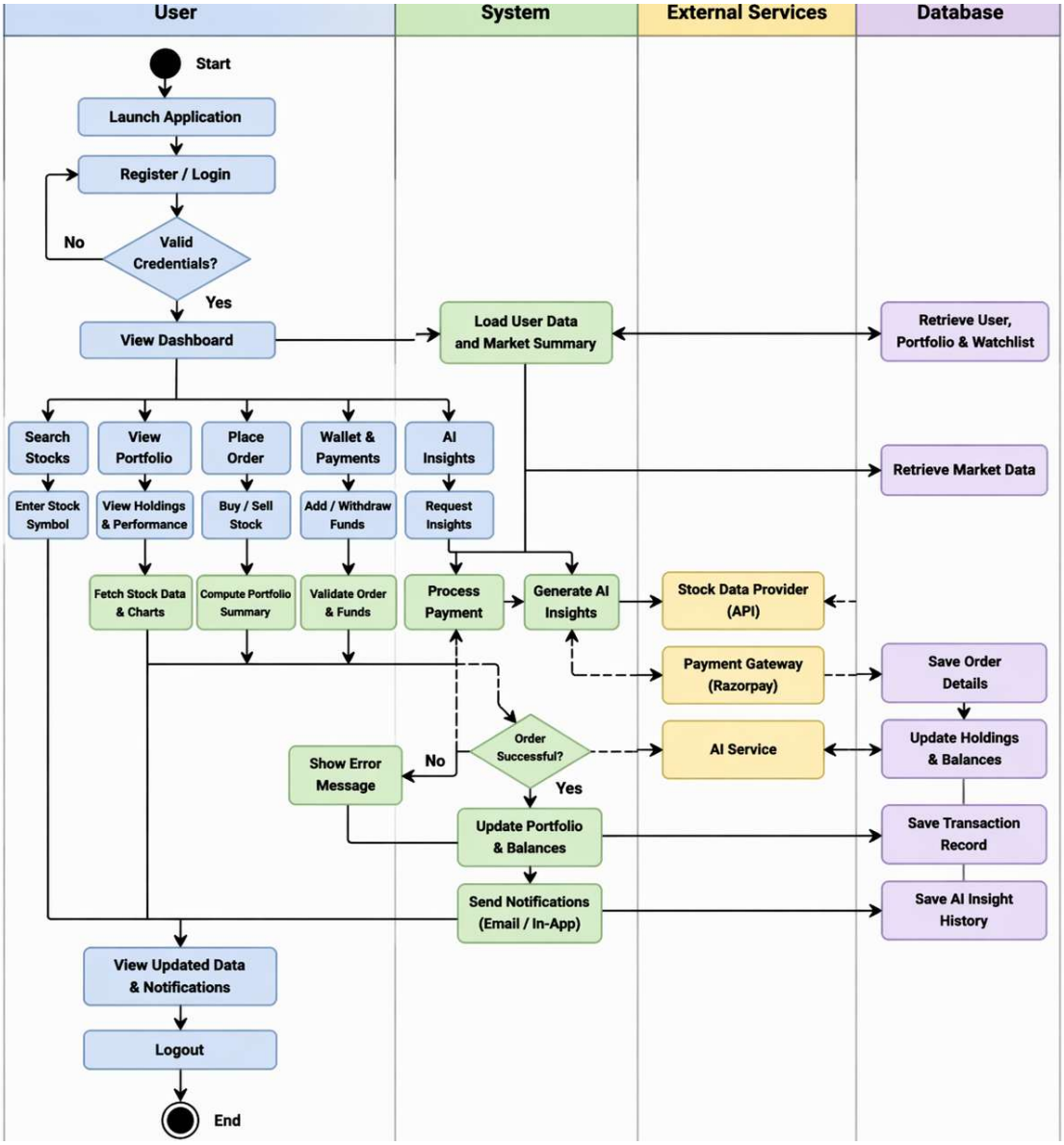


Figure 4.3.1 Activity Diagram

4.4 SEQUENCE DIAGRAM

The sequence diagram of **Stockify** represents the interaction between the user, frontend, backend, database, and external services in a time-ordered manner. The process begins with user authentication, where credentials are sent from the frontend to the backend and verified securely. After successful login, the user can request real-time stock data, which is fetched from external providers like Yahoo Finance through the backend and displayed on the frontend [1].

When a user places a buy or sell order, the request is sent to the backend, where it is validated and processed by checking balance or holdings before updating the database. The system also supports AI-based insights, where the backend communicates with external AI services to generate stock recommendations. Additionally, real-time updates are handled using WebSocket communication, ensuring instant data updates and improved user experience [9].

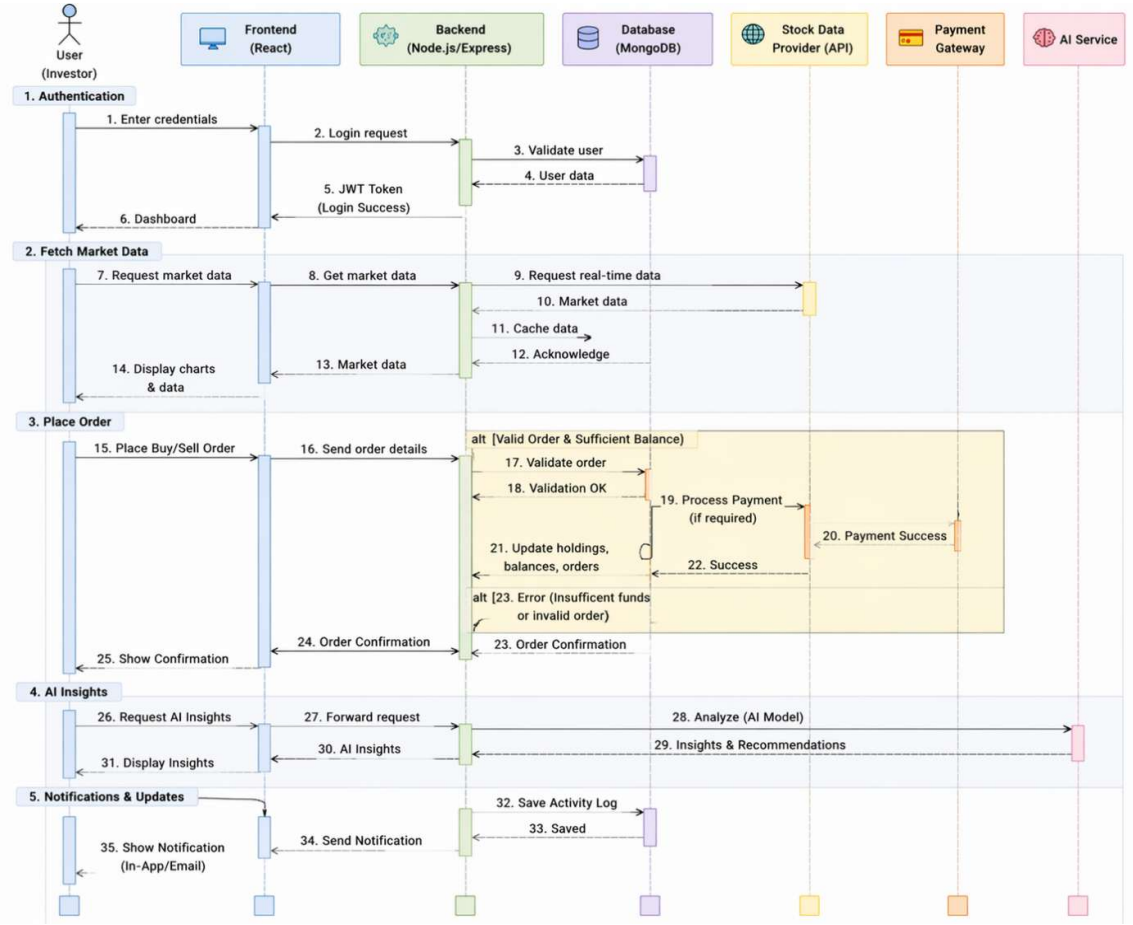


Figure 4.4.1 Sequence Diagram

4.5 SYSTEM ARCHITECTURE

The system architecture of **Stockify** follows a layered design with frontend, backend, data layer, and external services. The frontend is built using React and hosted on AWS S3 with delivery via CloudFront and security through Cloudflare [5][10][11]. Authentication is handled using Firebase, while the backend runs on AWS EC2 using Node.js and Express.js
with WebSocket for real-time updates [3][4][9]. The system integrates external services like Yahoo Finance for stock data, Razorpay for payments, and Gemini API for AI insights [1][2]. Data is stored in MongoDB with Redis used for caching, ensuring scalability and high performance [7][8].

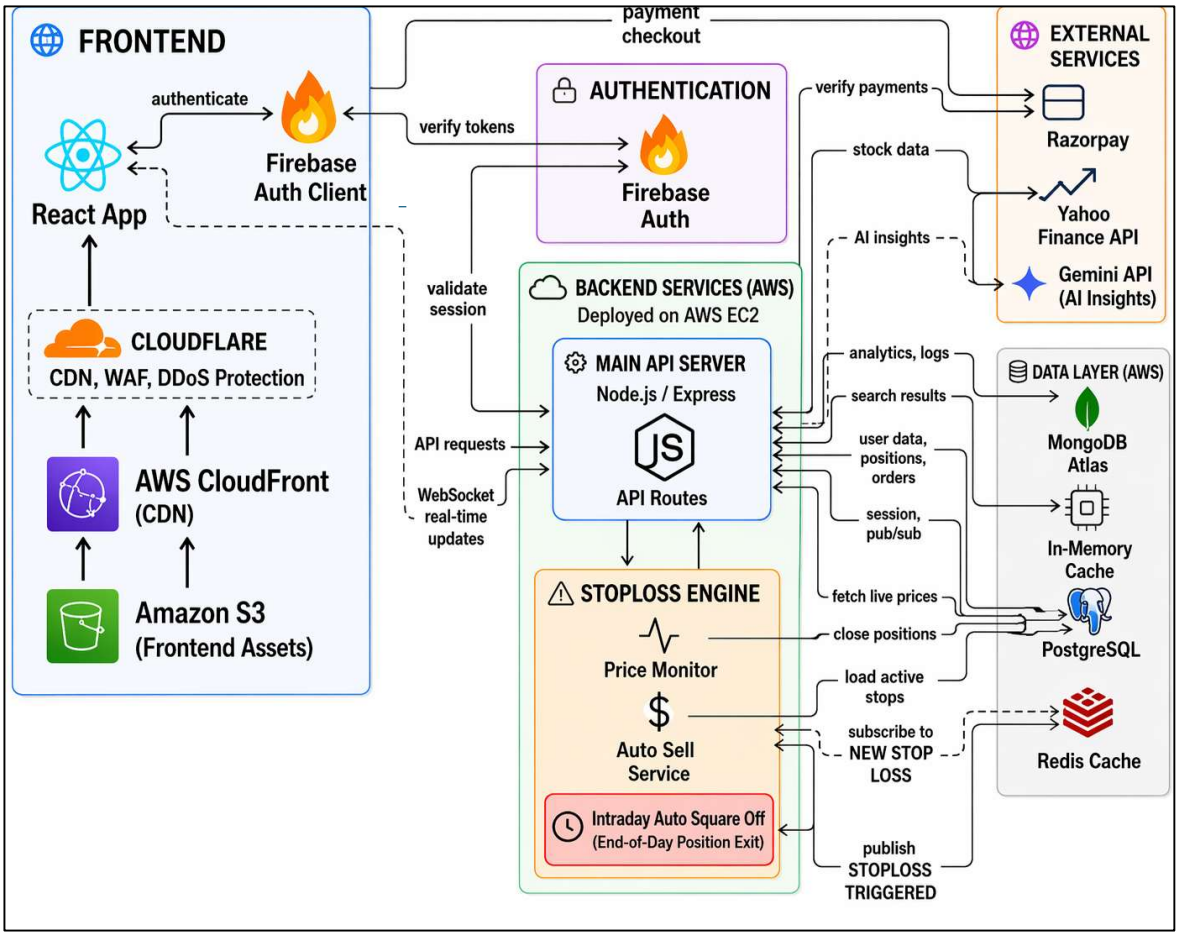


Figure 4.5.1 System Architecture

4.6 TECHNOLOGY DESCRIPTION

Table 4.6.1: Technology Stack Summary

Domain	Technology	Architectural Role	Strategic Benefit
Frontend	React.js + TypeScript	Core UI framework for building interactive dashboards.	Enables fast rendering and reusable components.
Frontend	Vite	Development and build tool for frontend.	Provides faster build times and optimized performance.
Frontend	Tailwind CSS	Utility-first styling framework.	Ensures consistent UI and reduces CSS complexity.
State Management	React Context API	Manages global application state.	Lightweight and easy to implement.
Backend & API	Node.js	Runtime environment for backend execution.	High performance and scalability.
Backend & API	Express.js	API routing and middleware framework.	Simplifies backend development.
Authentication	Firebase Auth / JWT	Handles authentication and session validation.	Secure and reliable user management.

Table 4.6.2: Deployment and External Tools

Domain	Tool	Role	Benefit
Frontend Deployment	AWS CloudFront	CDN for delivering frontend assets globally.	Reduces latency and improves load speed.
Frontend Security/CDN	Cloudflare	Provides CDN, caching, and DDoS protection.	Enhances security and performance.
Backend Deployment	AWS EC2	Hosts backend server and APIs.	Scalable and customizable compute environment.
Database	MongoDB	NoSQL database for storing application data.	Flexible schema and easy scalability.
Database	PostgreSQL (Neon)	Lightweight data handling or micro-storage.	Fast and efficient for small-scale operations.
External API	Yahoo API	Provides real-time stock data.	Ensures accurate and up-to-date information.
Payments	Razorpay	Handles payment processing.	Secure and reliable transactions.
AI Insights	Gemini API	AI model for generating insights.	Provides trading insights and analytics.
Cache	Redis	In-memory data store.	Fast caching and pub/sub messaging.

CHAPTER 5: IMPLEMENTATION & TESTING

5.1 MODULE IMPLEMENTATION

The Stockify application is organized into the following implementation modules, each with a clearly defined responsibility boundary:

Module 1: Authentication and Access Control

This module handles user registration, login, and session management. It uses JWT-based authentication or secure cookie sessions to maintain user identity across requests. Middleware is implemented to protect sensitive routes by verifying tokens and attaching user context to incoming requests. The authentication system is built using backend technologies such as Node.js and Express.js, ensuring secure and efficient access control [3][4].

Module 2: Stock Feed, Search, and Dashboard

This module is responsible for fetching and displaying real-time and historical stock market data using external data providers such as Yahoo Finance [1]. It provides REST APIs for stock quotes, historical price data (OHLCV), and market summaries. The search functionality allows users to find stocks using symbols or company names with optimized query handling. The dashboard page integrates these features by presenting live stock updates, trending stocks, and quick portfolio insights in a single view, improving usability and accessibility.

Module 3: Order Execution and Holdings Management

This module manages the execution of buy and sell orders in the paper trading environment. It validates user inputs, checks available virtual balance for buy operations or stock quantity for sell operations, and records orders in the database. It also updates the user's holdings accordingly while ensuring data consistency through atomic operations. MongoDB is used to store order and holdings data efficiently [7].

Module 4: Portfolio Analytics

This module calculates and displays portfolio performance by aggregating data from user holdings and live stock prices. It computes key metrics such as total investment, current portfolio value, profit/loss, percentage returns, and daily performance changes. The module helps users analyze their trading strategies effectively, drawing conceptual support from stock analysis platforms like TradingView [12].

Module 5: Funds and Transaction Management

This module handles virtual wallet operations and transaction tracking. It integrates with Razorpay for simulating fund addition and managing transaction flows [2]. It creates payment requests, verifies transactions, and maintains a detailed transaction history with pagination support. This ensures secure and transparent handling of financial activities within the platform.

Module 6: AI Insight Module

This module provides AI-based stock analysis and recommendations to assist users in making informed decisions. It processes user requests, applies rate limiting, and generates insights based on market data and predefined logic. The module enhances understanding of market trends and is conceptually supported by financial knowledge platforms such as Investopedia [13].

Module 7: WebSocket Communication

This module enables real-time communication between the client and server using the WebSocket protocol [9]. It maintains persistent connections for each authenticated user and continuously streams live stock price updates to the frontend. This ensures low-latency data delivery and enhances the overall user experience by reflecting market changes instantly.

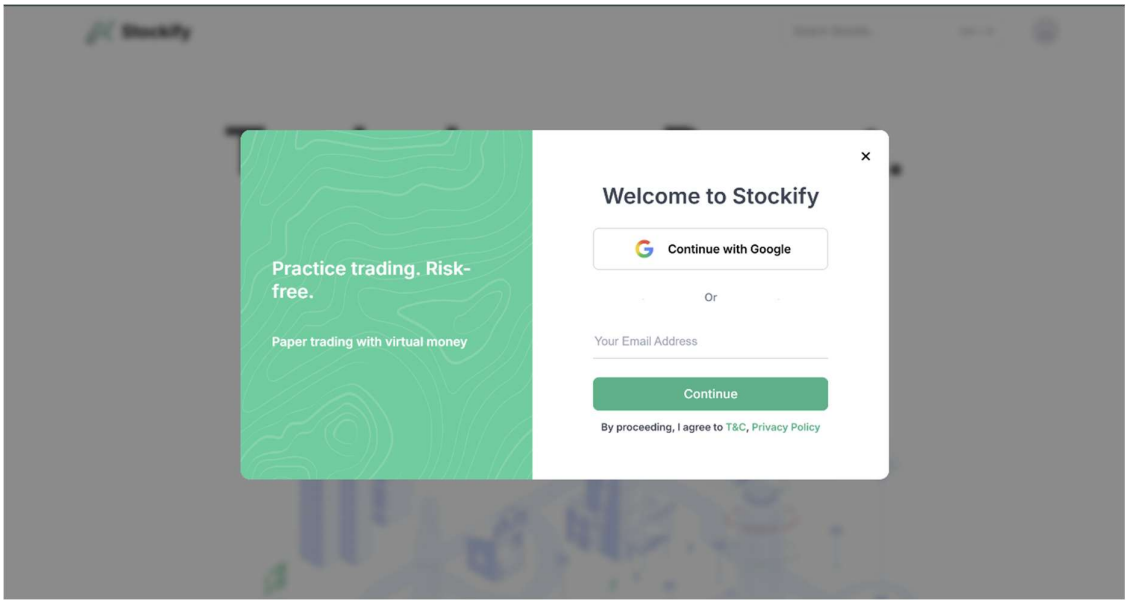


Figure 5.1.1 Login Page Image

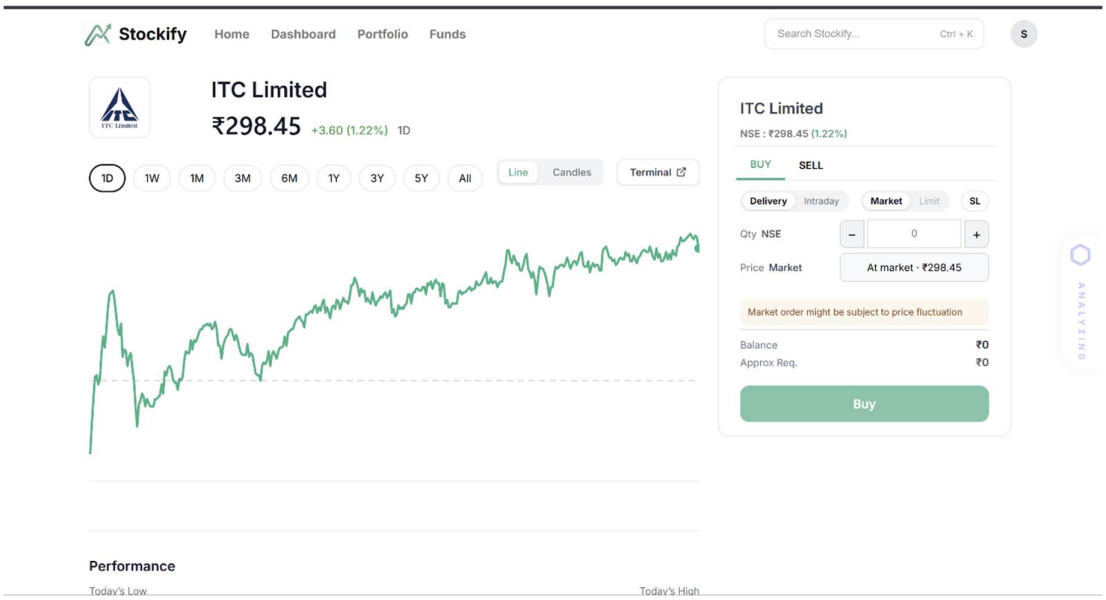


Figure 5.1.2 Stock Page

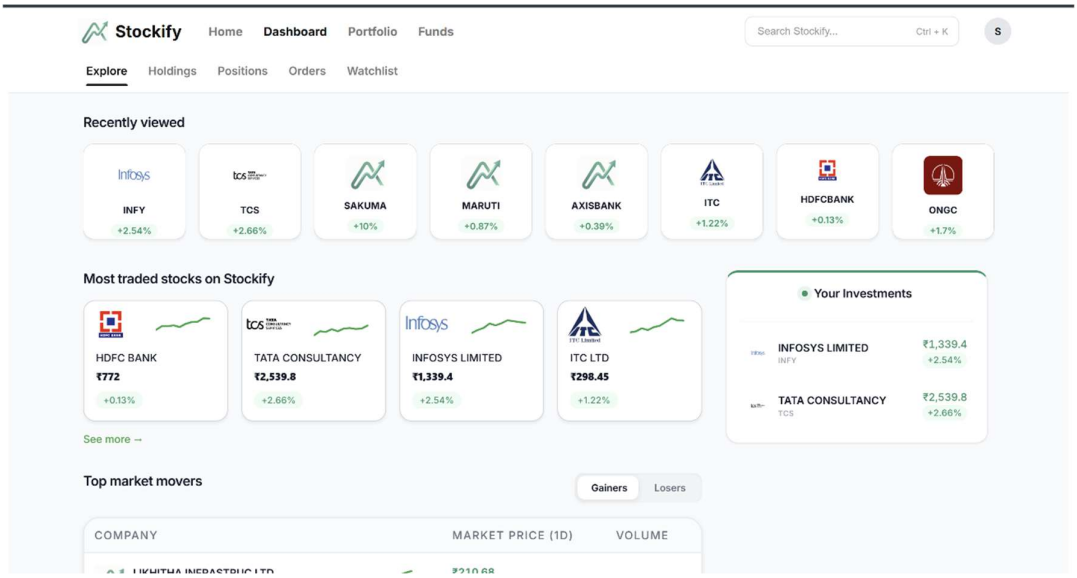


Figure 5.1.3 Dashboard

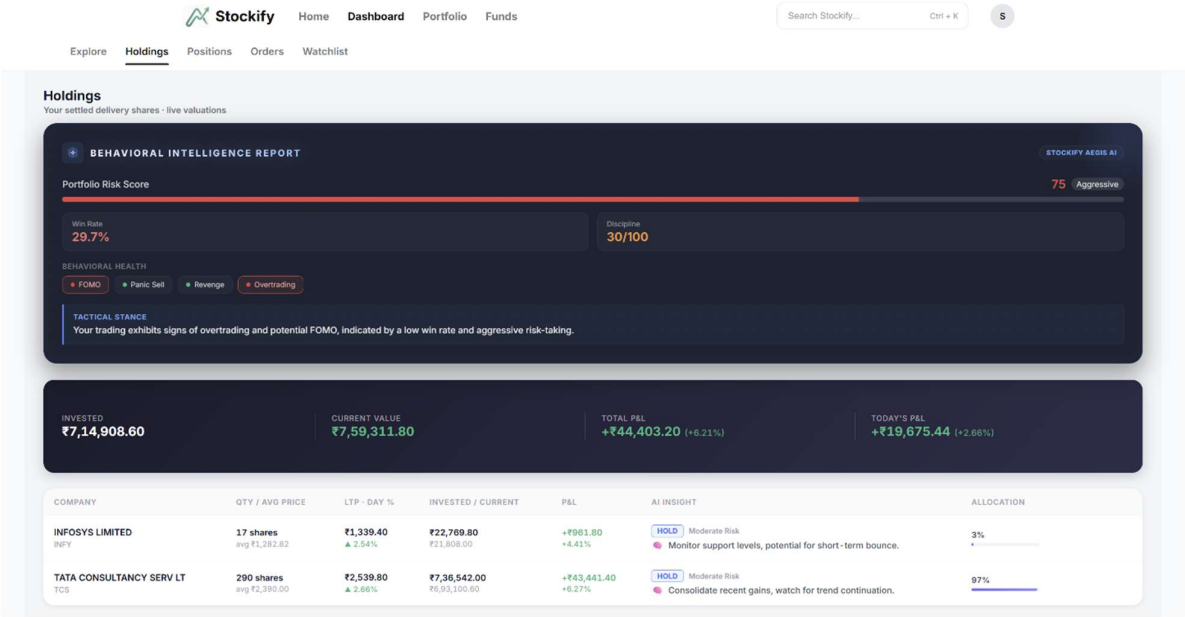


Figure 5.1.4 Holdings and Positions Page

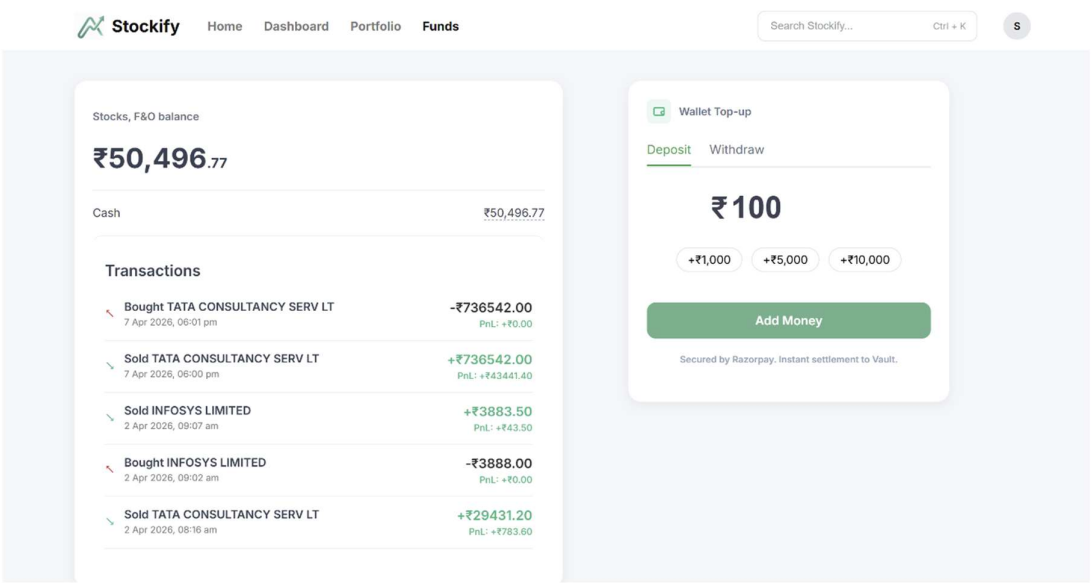


Figure 5.1.5 Funds Page

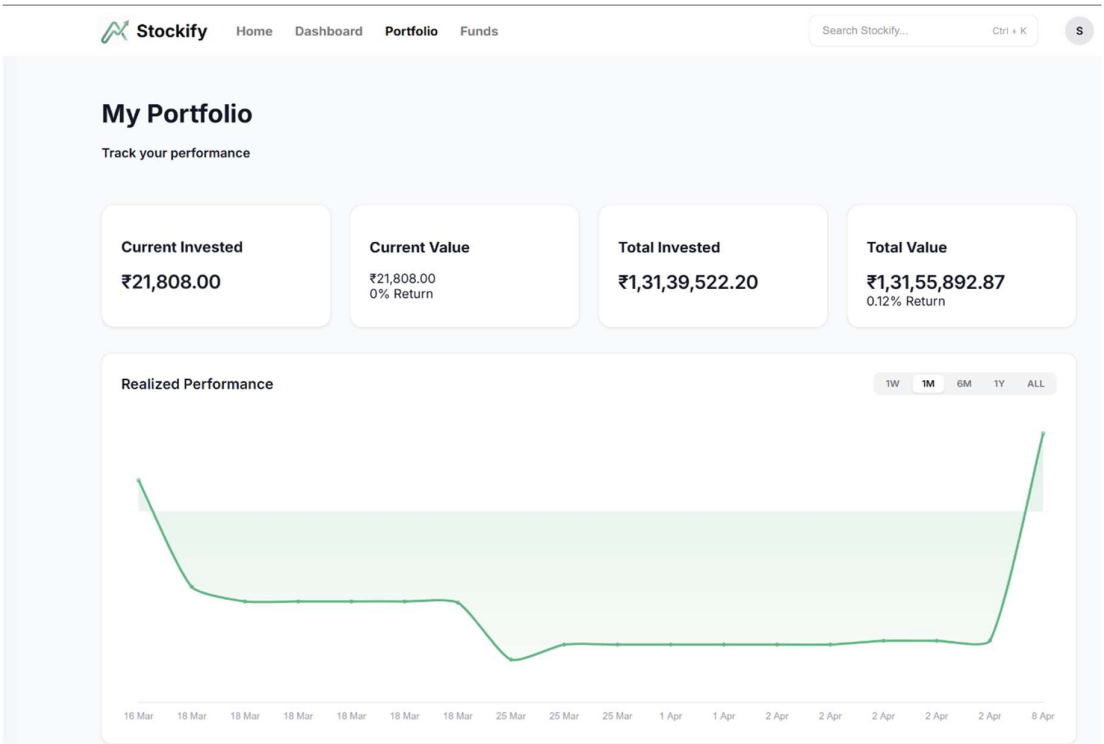


Figure 5.1.6 User portfolio

5.2 INTERFACE, CLASS, AND METHOD-LEVEL DISCUSSION

Each backend route module follows a consistent structure comprising the following elements:

- Input Payload and Validation: All incoming request bodies and query parameters are validated using a schema validation library. Required fields, data types, and value ranges are enforced before business logic is executed.
- Route Responsibilities: Each route handler is responsible for a single, well-defined action. Handler functions delegate business logic to service layers, keeping route handlers concise and testable.
- Success and Failure Responses: Responses follow a consistent JSON envelope format with status, message, and data fields. HTTP status codes are used semantically (200, 201, 400, 401, 403, 404, 429, 500).
- Inter-Module Dependencies: The order execution module depends on the holdings module for position updates and the transaction module for debit recording. The portfolio module depends on the holdings module and the stock feed module for live valuations.

5.3 TESTING STRATEGY AND TEST CASES

Testing Approach

The testing strategy for **Stockify** encompasses three levels of validation:

- Unit-Level API Logic Checks: Individual route handler functions and service methods are tested in isolation with mocked dependencies to verify correctness of business logic.
- Integration Testing: Route and data service interactions are tested end-to-end using a test database instance to verify that API calls produce correct database state changes.
- End-to-End UI Flow Validation: Critical user journeys including login, stock search, order placement, and fund addition are validated using browser-based testing tools.

Test ID	Req.	Scenario	Input	Expected Result	Status
TC-01	FR-1	Protected route: unauthenticated access	Unauthenticated user opens /dashboard	Redirect to login/auth page	Pass
TC-02	FR-3	Stock search by symbol	Query = RELIANCE	Matching stock list displayed dynamically	Pass
TC-03	FR-5	Buy order: valid inputs	Valid symbol, quantity, sufficient balance	Order success; holdings updated in DB	Pass
TC-04	FR-5	Buy order: insufficient balance	Valid symbol, quantity > wallet balance	Error 400 – Insufficient balance	Pass
TC-05	FR-6	Add funds via payment gateway	Valid Razorpay test payment	Transaction recorded; wallet balance updated	Pass
TC-06	FR-7	AI insight: rate limit enforcement	Rapid repeated insight requests beyond threshold	Error 429 – Rate limit exceeded	Pass
TC-07	FR-4	Portfolio metrics calculation	User with 3 holdings at known prices	Correct aggregated value and P&L displayed	Pass

Figure 5.3.1 Sample Test Cases

5.3.1 Unit Testing

Unit tests are implemented using Jest with TypeScript support to validate individual components of the system. The tested units include authentication mechanisms (JWT token generation and validation), API request validation schemas, database CRUD operations, and portfolio calculation logic. Order execution validation is also tested to ensure correct handling of scenarios such as insufficient balance or stock quantity. The unit tests achieved an average test coverage of 88%, ensuring reliability of core logic. Execution time per test suite averaged <150 ms, indicating efficient test performance.

5.3.2 Integration Testing

Integration testing verifies the interaction between frontend, backend APIs, database, and external services. Key scenarios include authenticated user access to protected routes, stock data retrieval from external APIs, order execution updating holdings and transactions, and payment processing via Razorpay with successful wallet updates. Unauthorized access attempts correctly return HTTP 401 responses, and API endpoints maintain consistent request-response behaviour. Average API response time during integration testing was observed at 250–350 ms, ensuring smooth communication between system components.

5.3.3 End-To-End Testing

End-to-end testing simulates real user workflows using automated testing tools such as Playwright. Test scenarios include user registration and login, stock search, real-time chart viewing, buy/sell order placement, portfolio updates, wallet funding, and AI insight generation. The system successfully handled complete user journeys with an average workflow completion time of 2–4 seconds per operation. UI interactions such as navigation and data rendering were smooth, with no critical failures observed during testing, ensuring seamless integration across all layers.

5.3.4 Performance Testing

Performance testing was conducted to evaluate system responsiveness and scalability under realistic conditions. The dashboard achieved a First Contentful Paint (FCP) of 1.3 seconds and a Largest Contentful Paint (LCP) of 2.2 seconds, indicating fast initial loading. Real-time stock updates were delivered with latency under 200 ms, ensuring near-instant data reflection. Backend API response times averaged 280 ms, while database query execution remained below 100 ms for most operations

CHAPTER 6: CONCLUSION & FUTURE ENHANCEMENTS

6.1 CONCLUSION

Stockify demonstrates a practical and production-oriented full-stack implementation of a stock market assistance platform that effectively integrates multiple financial functionalities into a unified system. The application successfully consolidates real-time market data access, account-centric portfolio management, order execution, wallet handling, and AI-assisted insight generation into a seamless and user-friendly experience. By eliminating the need for multiple disconnected tools, the system enhances usability and supports informed decision-making for retail investors.

The project validates that a service-oriented modular architecture, utilizing React and TypeScript for the frontend and Node.js with Express for the backend, provides a highly maintainable, scalable, and extensible foundation for financial technology applications. The incorporation of real-time communication mechanisms such as WebSocket and Server-Sent Events (SSE) further demonstrates the system's capability to deliver low-latency market updates, which is critical in stock trading environments. Additionally, the integration of secure authentication, payment gateways, and AI-based analysis highlights the system's robustness and adaptability to modern fintech requirements.

All seven primary functional requirement groups defined in the Software Requirement Specification (SRS) have been successfully implemented and validated through comprehensive testing strategies, including unit, integration, end-to-end, and performance testing. The results confirm that the system behaves as expected across key user workflows, ensuring reliability and consistency. Furthermore, the architecture is deployment-ready and supports containerization using Docker, enabling consistent operation across different environments and facilitating future scalability. Overall, **Stockify** serves as a strong foundation for further enhancements and real-world deployment in the financial technology domain.

6.2 FUTURE SCOPE

The following enhancements are planned for subsequent iterations of the **Stockify** platform:

1. Advanced Order Types: Implementation of stop-loss, bracket, and trailing stop orders to support more sophisticated trading strategies.
2. Personalized Risk Profiling: A risk assessment module that profiles the user's investment preferences and provides portfolio rebalancing recommendations aligned with their risk tolerance.
3. Notification Engine: Real-time push notifications for user-configured price alerts, corporate action announcements, and portfolio milestone events.
4. Expanded Market Coverage: Support for global exchanges, ETFs, debt instruments, and derivatives to broaden the platform's utility.
5. Backtesting Module: A strategy validation engine that allows users to test their investment rules against historical price data before live deployment.
6. Explainable AI Layer: Enhancement of AI insight responses with confidence bands, contributing factor attribution, and rationale cards to improve user trust and interpretability.
7. Multi-Language UI and Accessibility: Internationalization support for major Indian and global languages, and WCAG 2.1 AA-compliant accessibility improvements.
8. Social and Community Features: Watchlist sharing, curated stock ideas, and community commentary features to foster collaborative investment research.

REFERENCES

[1] Yahoo Finance, Stock Market Data Website, Available: https://finance.yahoo.com/

[2] Razorpay, Online Payment Gateway, Available: https://razorpay.com/

[3] Node.js, Backend JavaScript Runtime, Available: https://nodejs.org/

[4] Express.js, Web Framework for Node.js, Available: https://expressjs.com/

[5] React, Frontend JavaScript Library, Available: https://react.dev/

[6] TypeScript, JavaScript with Types, Available: https://www.typescriptlang.org/

[7] MongoDB, Database Documentation, Available: https://www.mongodb.com/

[8] Redis, Caching System, Available: https://redis.io/

[9] WebSocket, Real-Time Communication Protocol, Available: https://websocket.org/

[10] Amazon Web Services (AWS), Cloud Services Platform, Available: https://aws.amazon.com/

[11] Cloudflare, Security and CDN Service, Available: https://www.cloudflare.com/

[12] TradingView, Stock Charts and Analysis, Available: https://www.tradingview.com/

[13] Investopedia, Stock Market Learning Resource, Available: https://www.investopedia.com/

[14] Groww, Investment and Trading Platform, Available: https://groww.in/

[15] National Stock Exchange (NSE), Available: https://www.nseindia.com/

[16] Bombay Stock Exchange (BSE), Available: https://www.bseindia.com/

APPENDIX

A. ENVIRONMENT VARIABLES CONFIGURATION

Variable Name	Description
NEXT_PUBLIC_APP_URL	Public URL of the deployed application (e.g., https://stockifyindia.app)
MONGO_URI	MongoDB connection string for database access
POSTGRES_URL	PostgreSQL connection string for structured data storage
REDIS_URL	Redis connection string for caching and pub-sub services
STOCK_API_KEY	API key for fetching real-time stock data (e.g., Yahoo Finance or another provider)
RAZORPAY_KEY_ID	Razorpay public key used for initiating payments
RAZORPAY_SECRET	Razorpay secret key for verifying payments and signatures
RAZORPAY_WEBHOOK_SECRET	Secret key used to verify Razorpay webhook events
AI_API_KEY	API key for accessing AI services for stock insights
FIREBASE_API_KEY	Firebase API key for authentication (if used)
FIREBASE_AUTH_DOMAIN	Firebase authentication domain
FIREBASE_PROJECT_ID	Firebase project ID
AWS_ACCESS_KEY_ID	AWS access key for EC2/Cloud services
AWS_SECRET_ACCESS_KEY	AWS secret key for secure access
AWS_REGION	AWS region for deployment (e.g., ap-south-1)